UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN

SISTEMA DE UNIVERSIDAD ABIERTA

Alumno: Emanuel Marzano Villalobos

Programación De Dispositivos Móviles

Cristian Cardoso Arellano

Grupo 9697

m5_01

21/10/2026

Enlace a repositorio de la actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M5_01

1. Introducción

La secuencia de Fibonacci es una serie de números en la que cada número es la suma de los dos anteriores: 1, 1, 2, 3, 5, 8, 13, 21… Es un ejemplo clásico en programación porque permite practicar estructuras repetitivas y, además, muestra un problema importante: si no se tiene cuidado, la computadora puede repetir muchas veces los mismos cálculos.

En esta actividad se desarrolla una aplicación Android en lenguaje Java que muestra la secuencia de Fibonacci en pantalla. La aplicación tiene un texto (TextView) que muestra el valor actual y dos botones: uno para avanzar al siguiente número y otro para retroceder al anterior. Los números se calculan con la estructura de control for y se aplica la técnica de memoization, que consiste en guardar los resultados ya calculados para no tener que calcularlos otra vez.

A diferencia de las actividades anteriores, esta es la primera en la que la aplicación tiene una interfaz con la que el usuario interactúa, por lo que también se trabaja la unión entre el diseño en XML y el código en Java.

2. Objetivo

Comprender y practicar la estructura de control for mediante el cálculo de la secuencia de Fibonacci para cualquier número “n”, aplicando la técnica de memoization y desplegando el resultado en un TextView que se actualiza con dos botones (avance y retroceso) definidos con recursos XML.

3. Conceptos utilizados

3.1. La secuencia de Fibonacci

Cada número de la secuencia se obtiene sumando los dos anteriores. Si F(n) representa el número que está en la posición n:

```
F(0) = 0
F(1) = 1
F(n) = F(n - 1) + F(n - 2)
```

Por ejemplo, F(5) = F(4) + F(3) = 3 + 2 = 5. Así, a partir de la posición 1 la secuencia queda: 1, 1, 2, 3, 5, 8, 13…, que es justamente lo que pide la actividad en cada clic.

3.2. La estructura de control for

Un ciclo for repite un bloque de código un número determinado de veces. Tiene tres partes dentro del paréntesis, separadas por punto y coma:

```
for (inicio; condición; incremento) {
    // código que se repite
}
```

- **`inicio`**: se ejecuta una sola vez al empezar; normalmente crea un contador, por ejemplo int i = 2.
- **`condición`**: se revisa antes de cada vuelta; mientras sea verdadera, el ciclo continúa.
- **`incremento`**: se ejecuta al final de cada vuelta; normalmente i++, que suma 1 al contador.

3.3. ¿Qué es la memoization?

La memoization es una técnica que consiste en guardar en memoria los resultados que ya se calcularon, para que la próxima vez que se necesiten se lean directamente en lugar de calcularlos otra vez.

> **Una comparación para entenderlo**
>
> Imagina que alguien te pregunta cuánto es 3,452 × 17. Lo calculas con calma y anotas el resultado en una libreta. Si cinco minutos después te vuelven a preguntar lo mismo, ya no haces la operación: solo lees la libreta. La memoization es esa libreta.
>
> En esta aplicación, la "libreta" es una lista llamada `memo`. Cada vez que se calcula un número de Fibonacci, se guarda en la lista. Cuando el usuario retrocede, los números ya están guardados y no se vuelve a calcular nada.

Esto es especialmente importante en Fibonacci, porque cada número depende de los dos anteriores. Sin memoization, para obtener un número grande habría que recalcular una y otra vez los mismos valores pequeños, y el programa se volvería muy lento.

3.4. Unión entre el diseño XML y el código Java

En Android, la pantalla se define en un archivo XML (el diseño) y el comportamiento en un archivo Java (la lógica). Para que trabajen juntos se usan tres elementos:

- **`android:id`**: en el XML, le da un nombre único a cada elemento de la pantalla, por ejemplo @+id/btnAvanzar.
- **`findViewById`**: en Java, busca ese elemento por su nombre para poder usarlo desde el código.
- **`setOnClickListener`**: en Java, indica qué debe hacer la aplicación cuando el usuario da clic en un botón.

4. Desarrollo

4.1. Creación del proyecto

Se creó el proyecto M5_01 con la plantilla Empty Views Activity y el lenguaje Java. Se utilizan tres archivos: strings.xml (los textos), activity_main.xml (el diseño) y MainActivity.java (la lógica).

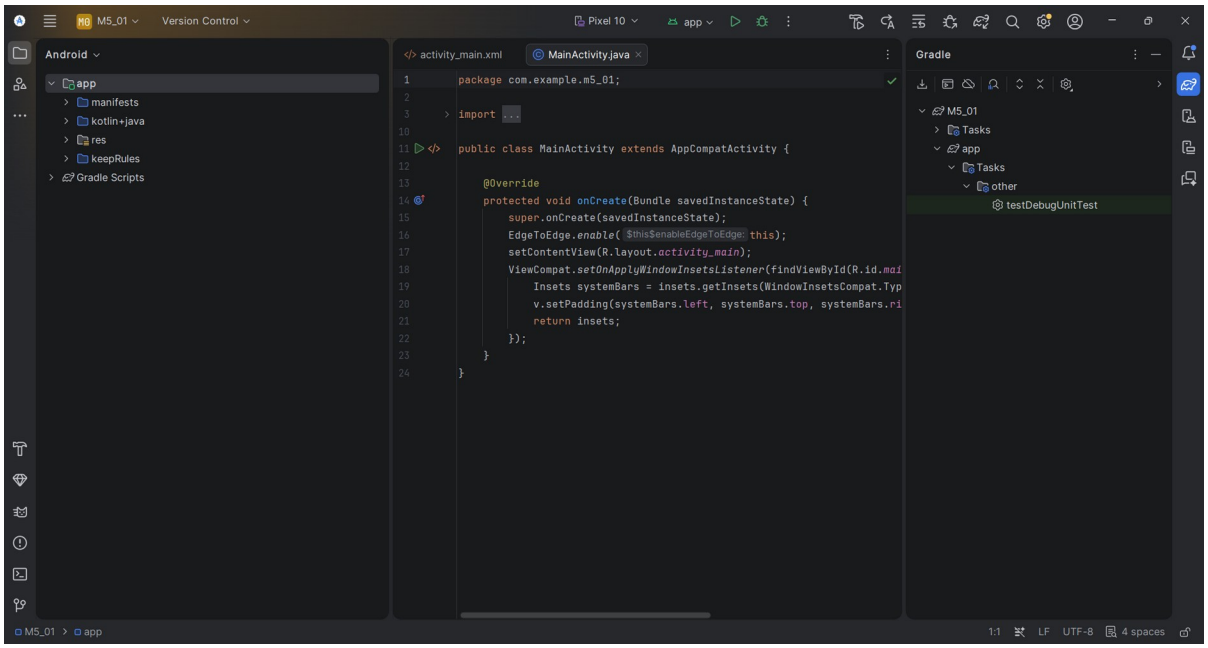


Figura 1. Proyecto M5_01 creado.

4.2. Recursos de texto: strings.xml

La actividad pide utilizar recursos XML. En Android, la forma correcta de manejar los textos es guardarlos en el archivo res/values/strings.xml en lugar de escribirlos directamente en el diseño. Así todos los textos quedan en un solo lugar y es más fácil modificarlos o traducir la aplicación a otro idioma.

```
<resources>
    <string name="app_name">M5_01</string>
    <string name="titulo">Secuencia Fibonacci</string>
    <string name="mensaje_inicial">Presiona Avanzar</string>
    <string name="boton_avanzar">Avanzar</string>
    <string name="boton_retroceder">Retroceder</string>
</resources>
```

- **`<string name="…">`**: cada línea define un texto. El atributo name es el nombre con el que se usa desde el diseño o el código; lo que está entre las etiquetas es el texto que se muestra.

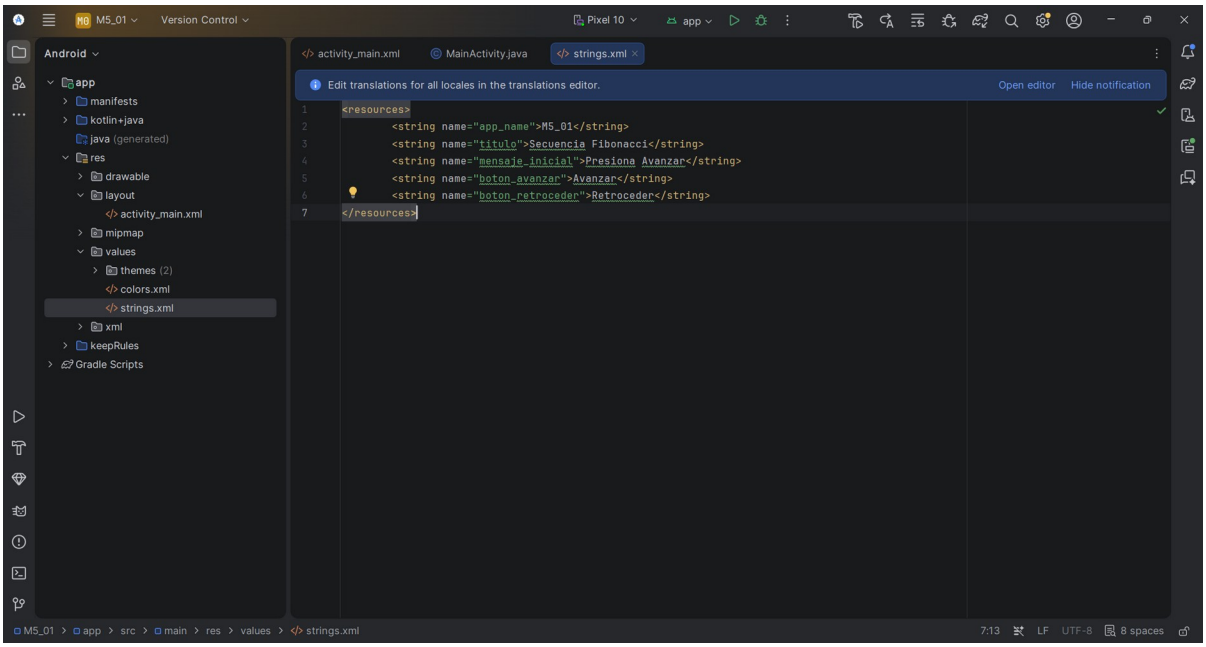


Figura 2. Textos de la aplicación en strings.xml.

4.3. Diseño de la pantalla: activity_main.xml

El diseño se reemplazó por uno basado en LinearLayout, que acomoda los elementos uno tras otro. Contiene un título, el TextView que muestra el número y dos botones.

```
<LinearLayout ...
    android:id="@+id/main"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvTitulo"
        android:text="@string/titulo"
        android:textSize="24sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvValor"
        android:text="@string/mensaje_inicial"
        android:textSize="48sp" />

    <LinearLayout
        android:orientation="horizontal">
        <Button
            android:id="@+id/btnRetroceder"
            android:text="@string/boton_retroceder" />
        <Button
            android:id="@+id/btnAvanzar"
            android:text="@string/boton_avanzar" />
    </LinearLayout>
</LinearLayout>
```

(Se muestran solo los atributos principales; las medidas de ancho, alto y márgenes de cada elemento se observan en la figura 3.)

- **`LinearLayout vertical`**: el contenedor principal coloca los elementos uno debajo de otro; gravity="center" los centra en la pantalla.
- **`TextView tvTitulo`**: muestra el título “Secuencia Fibonacci”.
- **`TextView tvValor`**: es el elemento que pide la actividad: muestra el valor actual de la secuencia. Al iniciar muestra “Presiona Avanzar” y tiene letra grande (48sp).
- **`LinearLayout horizontal`**: un segundo contenedor, dentro del primero, que coloca los dos botones uno junto al otro.
- **`Button btnRetroceder / btnAvanzar`**: los dos botones que pide la actividad.
- **`@string/…`**: toma el texto del archivo strings.xml; esto es el uso de recursos XML.
- **`android:id="@+id/…"`**: nombre de cada elemento para encontrarlo desde Java. El “+” indica que el nombre se crea en ese momento.
- **`dp y sp`**: unidades que se adaptan al tamaño y densidad de cada pantalla: dp para espacios y sp para el tamaño de letra.

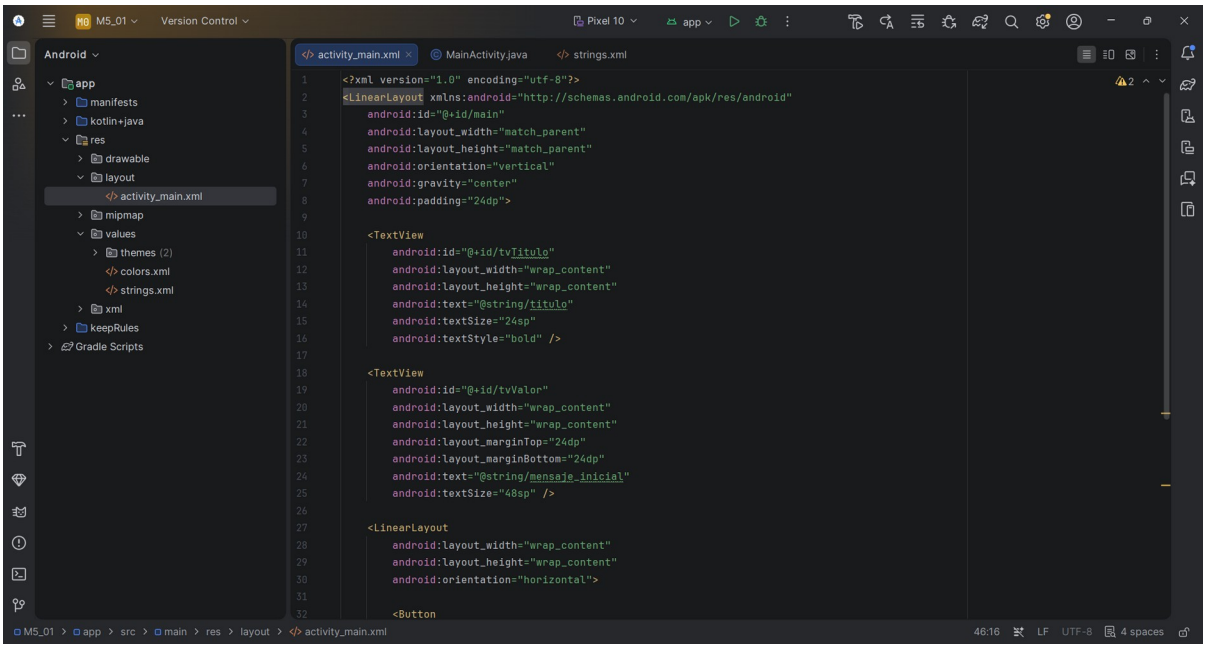


Figura 3. Código del diseño en activity_main.xml.

4.4. Lógica de la aplicación: MainActivity.java

Variables de la clase

```
private static final String TAG = "M5_01";
private static final int MAX_POSICION = 92;
private final List<Long> memo = new ArrayList<>();
private int posicion = 0;
private TextView tvValor;
```

- **`TAG`**: etiqueta para filtrar los mensajes en Logcat.
- **`MAX_POSICION = 92`**: límite de la secuencia. Los números de Fibonacci crecen muy rápido y F(93) ya no cabe en una variable de tipo long (el número entero más grande de Java para este uso). Con este límite se evita mostrar un resultado incorrecto.
- **`List<Long> memo`**: la lista donde se guardan los números ya calculados: es la memoria de la memoization. Es una ArrayList, una lista que puede crecer conforme se agregan elementos.
- **`posicion`**: indica en qué número de la secuencia se encuentra el usuario. Empieza en 0, que significa que todavía no se ha presionado ningún botón.
- **`tvValor`**: guardará la referencia al TextView del diseño para poder cambiar su texto.

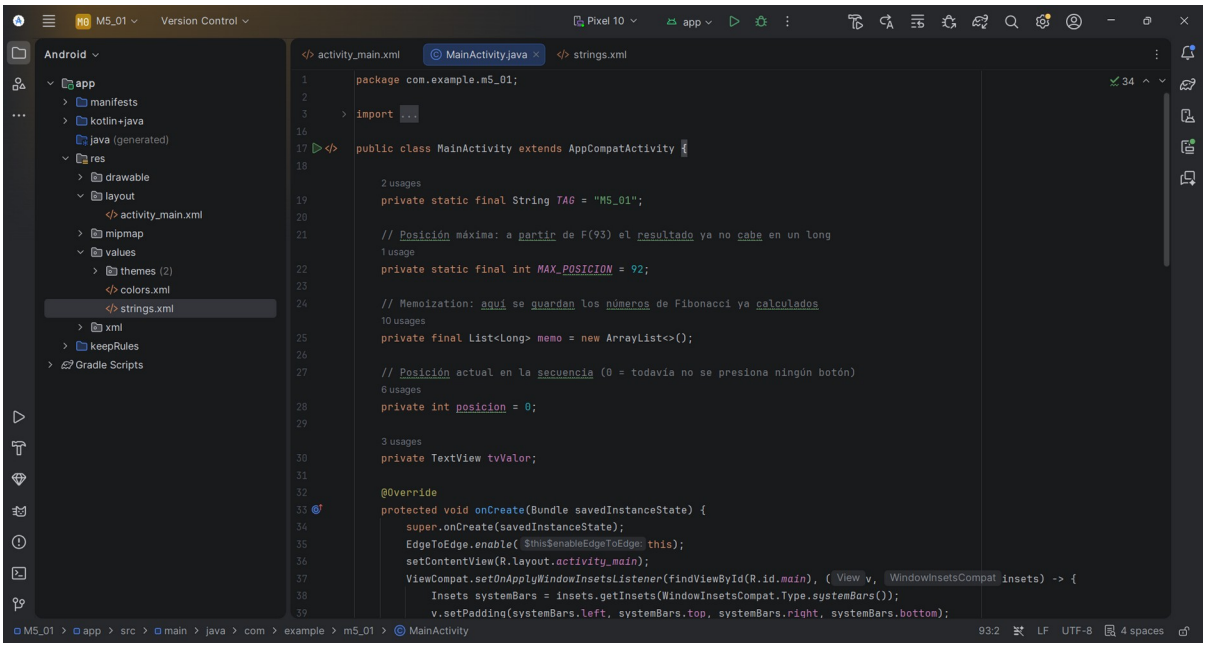


Figura 4. Variables de la clase MainActivity.

Método onCreate()

```
memo.add(0L);
memo.add(1L);

tvValor = findViewById(R.id.tvValor);
Button btnAvanzar = findViewById(R.id.btnAvanzar);
Button btnRetroceder = findViewById(R.id.btnRetroceder);

btnAvanzar.setOnClickListener(v -> avanzar());
btnRetroceder.setOnClickListener(v -> retroceder());
```

- **`memo.add(0L); memo.add(1L);`**: guarda los dos valores iniciales de la secuencia, F(0) = 0 y F(1) = 1, que no se pueden calcular a partir de otros. La “L” indica que son números de tipo long.
- **`findViewById(R.id.tvValor)`**: conecta el código con el TextView del diseño usando su id. Lo mismo se hace con los dos botones.

- **`setOnClickListener(v -> avanzar())`**: indica que al dar clic en el botón se ejecute el método avanzar(). La expresión v -> avanzar() es una lambda: una forma corta de escribir “cuando ocurra el clic, haz esto”. El v representa el botón que se presionó.

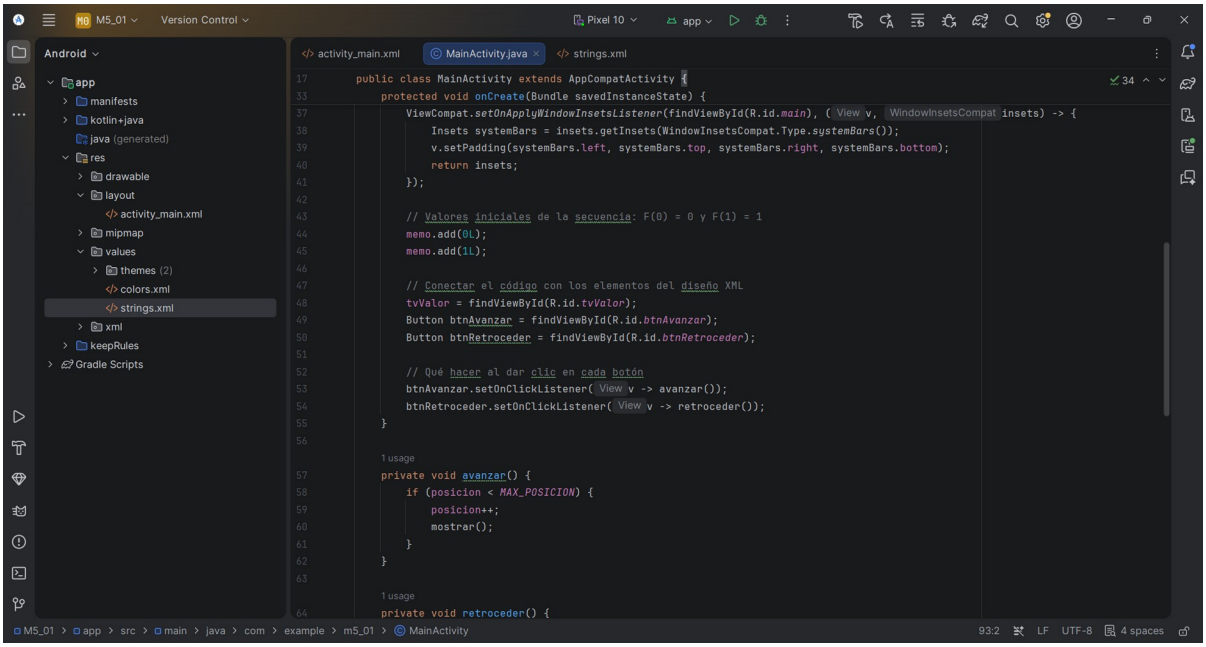


Figura 5. Método onCreate(): valores iniciales y conexión con el diseño.

Métodos avanzar(), retroceder() y mostrar()

```
private void avanzar() {
    if (posicion < MAX_POSICION) {
        posicion++;
        mostrar();
    }
}

private void retroceder() {
    if (posicion > 1) {
        posicion--;
        mostrar();
    } else {
        posicion = 0;
        tvValor.setText(R.string.mensaje_inicial);
    }
}

private void mostrar() {
    long valor = fibonacci(posicion);
    tvValor.setText(String.valueOf(valor));
}
```

- **`avanzar()`**: si no se ha llegado al límite, suma 1 a la posición (posicion++) y muestra el nuevo número.

- **`retroceder()`**: si la posición es mayor que 1, resta 1 (posicion--) y muestra el número anterior. Si ya está en la primera posición, regresa al estado inicial y vuelve a mostrar “Presiona Avanzar”.
- **`mostrar()`**: pide el número de Fibonacci de la posición actual y lo coloca en el TextView. String.valueOf convierte el número en texto, porque setText solo acepta texto.

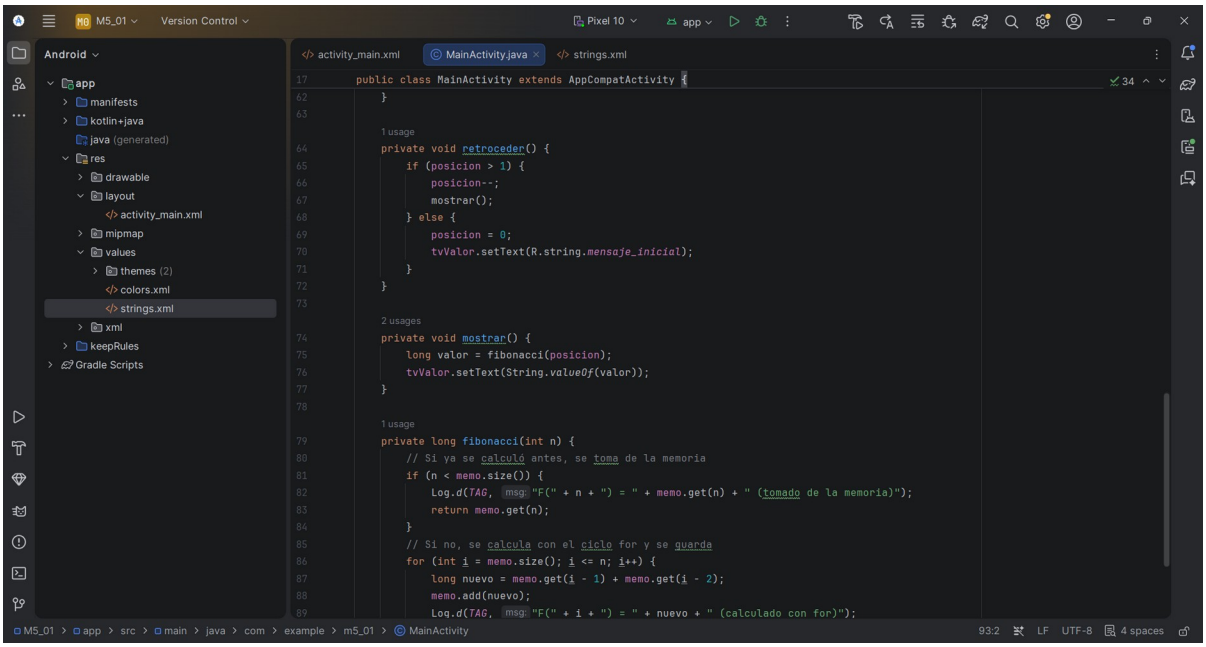


Figura 6. Métodos retroceder(), mostrar() y el inicio de fibonacci().

Método fibonacci(): el ciclo for con memoization

Este es el método más importante de la actividad, porque contiene el ciclo for y la memoization:

```
private long fibonacci(int n) {
    // Si ya se calculó antes, se toma de la memoria
    if (n < memo.size()) {
        Log.d(TAG, "F(" + n + ") = " + memo.get(n) + " (tomado de la
memoria)");
        return memo.get(n);
    }
    // Si no, se calcula con el ciclo for y se guarda
    for (int i = memo.size(); i <= n; i++) {
        long nuevo = memo.get(i - 1) + memo.get(i - 2);
        memo.add(nuevo);
        Log.d(TAG, "F(" + i + ") = " + nuevo + " (calculado con for)");
    }
    return memo.get(n);
}
```

- **`if (n < memo.size())`**: memo.size() indica cuántos números hay guardados. Si n es menor, significa que F(n) ya está en la lista, así que se regresa directamente con memo.get(n), sin calcular nada. Esta es la memoization.

- **`for (int i = memo.size(); i <= n; i++)`**: si el número no está guardado, el ciclo empieza justo en la primera posición que falta (memo.size()) y avanza de uno en uno hasta llegar a n. Así nunca se recalcula un número que ya existe.
- **`memo.get(i - 1) + memo.get(i - 2)`**: aplica la fórmula de Fibonacci: suma los dos números anteriores, que ya están guardados en la lista.
- **`memo.add(nuevo)`**: guarda el número recién calculado para usarlo después.
- **`Log.d(…)`**: imprime en Logcat si el número se calculó con el for o se tomó de la memoria. Sirve como evidencia de que la memoization funciona.

Para ver el ciclo en acción, supongamos que la lista tiene [0, 1] y se pide fibonacci(5) de una sola vez. El ciclo empieza en i = 2 (porque memo.size() es 2) y da cuatro vueltas:

Vuelta (i)	Operación	Resultado	memo después de la vuelta
i = 2	memo.get(1) + memo.get(0) = 1 + 0	1	[0, 1, 1]
i = 3	memo.get(2) + memo.get(1) = 1 + 1	2	[0, 1, 1, 2]
i = 4	memo.get(3) + memo.get(2) = 2 + 1	3	[0, 1, 1, 2, 3]
i = 5	memo.get(4) + memo.get(3) = 3 + 2	5	[0, 1, 1, 2, 3, 5]

Al terminar, F(5) = 5 queda guardado junto con todos los anteriores. Si después se pide fibonacci(3), la condición del if se cumple y el valor se toma de la lista sin entrar al ciclo.

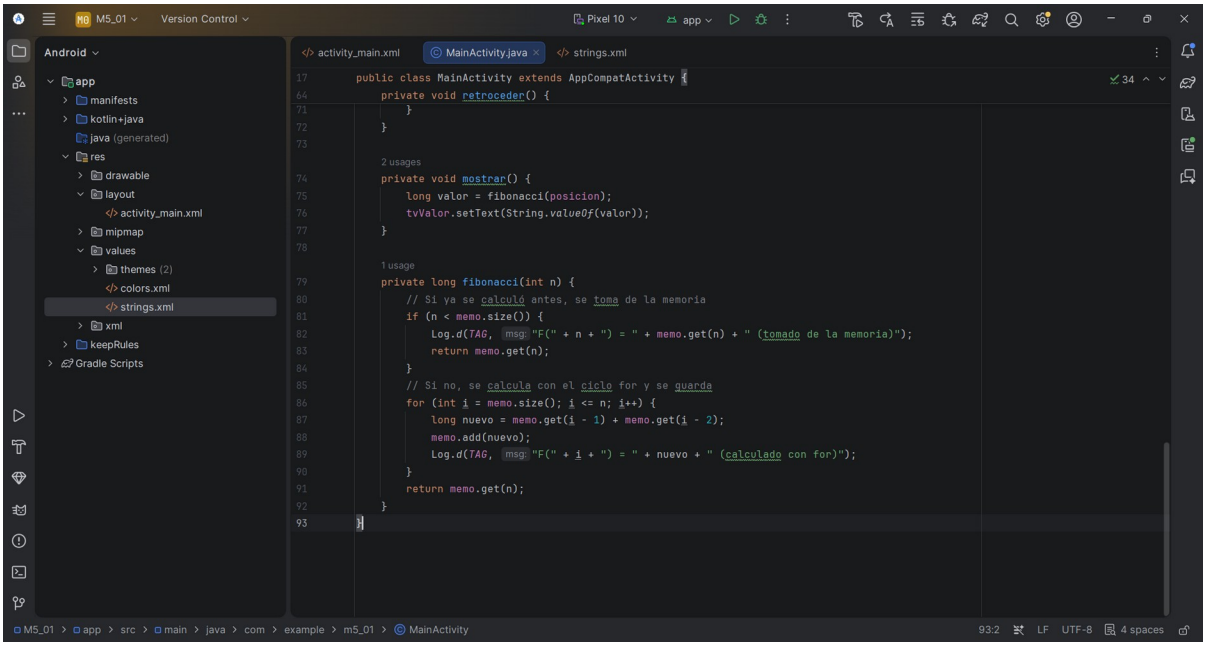


Figura 7. Método fibonacci() con el ciclo for y la memoization.

4.5. Pruebas y resultados

La aplicación se ejecutó en el emulador Pixel 10 (Android 17, API 37). Se presionó seis veces el botón Avanzar y después dos veces Retroceder, mientras se observaba Logcat con el filtro tag:M5_01.

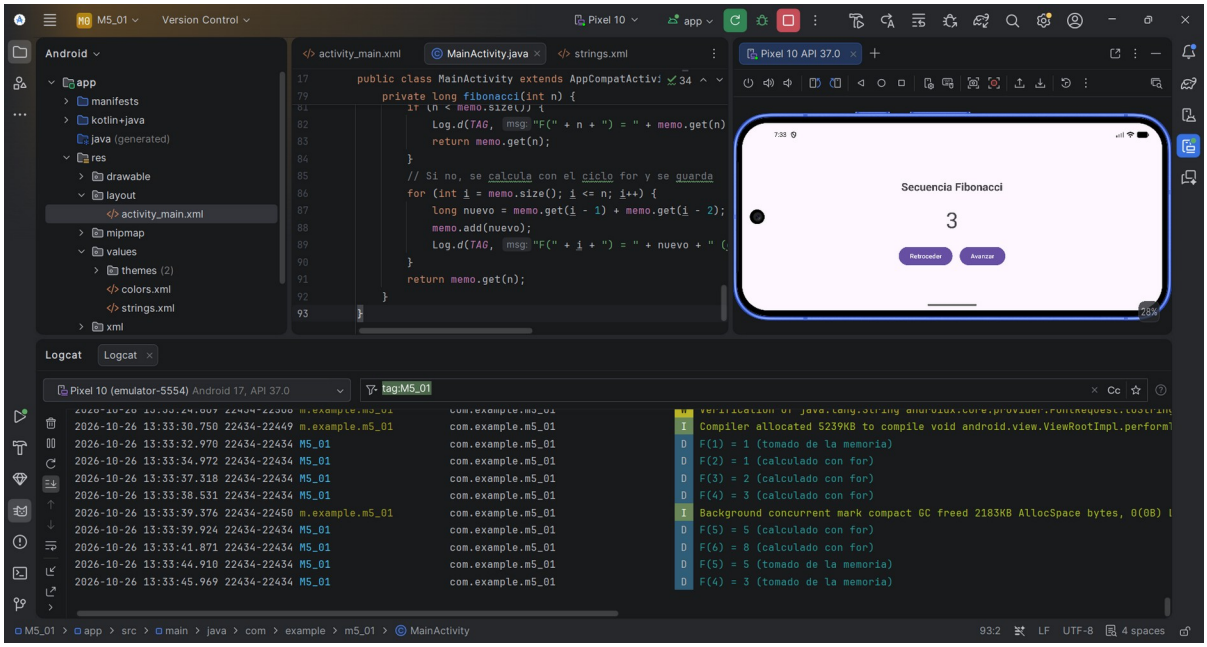


Figura 8. Aplicación en ejecución (valor 3) y mensajes de Logcat.

Descripción: En el emulador se observa el título “Secuencia Fibonacci”, el valor actual (3) y los dos botones. En Logcat se comprueba el funcionamiento de la memoization: F(1) se tomó de la memoria porque ya estaba guardado desde el inicio; de F(2) a F(6) cada número se calculó con el ciclo for y se guardó; al retroceder, F(5) y F(4) se tomaron de la memoria sin volver a calcularse. El emulador aparece en posición horizontal porque se había girado en una práctica anterior, lo que no afecta el funcionamiento de la aplicación.

La siguiente tabla resume lo que ocurrió en cada clic:

Acción	posicion	En pantalla	¿De dónde salió?	Contenido de memo
Inicio	0	Presiona Avanzar	—	[0, 1]
Avanzar #1	1	1	Memoria	[0, 1]
Avanzar #2	2	1	Calculado con for	[0, 1, 1]
Avanzar #3	3	2	Calculado con for	[0, 1, 1, 2]
Avanzar #4	4	3	Calculado con for	[0, 1, 1, 2, 3]
Avanzar #5	5	5	Calculado con for	[0, 1, 1, 2, 3, 5]
Avanzar #6	6	8	Calculado con for	[0, 1, 1, 2, 3, 5, 8]
Retroceder	5	5	Memoria	sin cambios
Retroceder	4	3	Memoria	sin cambios

Los valores mostrados en cada clic (1, 1, 2, 3, 5, 8) coinciden con los que pide la actividad.

Código fuente de esta actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M5-01

5. Conclusión

De todas las actividades, esta fue de las más interesantes y también de las más complicadas. Ya había probado la secuencia de Fibonacci en lenguaje C, pero el planteamiento siempre se me dificulta: pensar cómo sumar el número anterior con el actual, obtener el resultado y volver a sumarlo con el anterior para que la secuencia crezca. A eso se sumó entender cómo se crean los botones, cómo se conectan con el código y cómo funciona la memoria en este lenguaje.

Con la memoization entendí que los números nuevos y anteriores quedan guardados para ir sumando, y que si regreso a uno anterior es solo como regresar, sin hacer un cálculo nuevo. Lo comprobé en Logcat: al avanzar, cada número aparecía como calculado con el ciclo for, y al retroceder, como tomado de la memoria.

Además, por fin la aplicación tenía botones que respondían a mis clics y mostraban cada número en la pantalla, algo que no había pasado en las actividades anteriores. También me llamó la atención que los números de Fibonacci crecen tan rápido que a partir de la posición 93 ya no caben en una variable de tipo long, así que tuve que poner un límite.

6. Referencias

Alexis Martinovic. (2018, 3 de julio). *Todo sobre los píxeles por pulgada (PPI), ¿qué son los DPI? | Andro UY* [Video]. YouTube. https://www.youtube.com/watch?v=o9uyAkZrMD0

Chio Code. (2022, 1 de febrero). *Programación dinámica | Memoización | Explicado paso a paso* [Video]. YouTube. https://www.youtube.com/watch?v=e5zaZEfHsIs

Cómo agregar botones a tu app. (s. f.). Android Developers. https://developer.android.com/develop/ui/views/components/button?hl=es-419

Cristian Henao. (2017, 5 de mayo). *40. Cómo usar el archivo String.xml en Android* [Video]. YouTube. https://www.youtube.com/watch?v=EdtlQc1RTVo

Cuadrículas y unidades. (s. f.). Android Developers. https://developer.android.com/design/ui/mobile/guides/layout-and-content/grids-and-units?hl=es-419

Descripción general de los eventos de entrada. (s. f.). Android Developers. https://developer.android.com/develop/ui/views/touch-and-input/input-events?hl=es-419

Google. (s. f.-a). *Create a linear layout*. Android Developers. https://developer.android.com/develop/ui/views/layout/linear

Google. (s. f.-b). *Long*. Android Developers. https://developer.android.com/reference/java/lang/Long

Google. (s. f.-c). *String resources*. Android Developers. https://developer.android.com/guide/topics/resources/string-resource

Lecture 19: Dynamic programming I: Fibonacci, shortest paths. (s. f.). MIT OpenCourseWare. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/resources/lecture-19-dynamic-programming-i-fibonacci-shortest-paths/

OpenWebinars. (2016, 4 de febrero). *Diferencias entre PX, DP y SP en Android* [Video]. YouTube. https://www.youtube.com/watch?v=0CyXsh3617s

Oracle. (s. f.). *The for statement*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/nutsandbolts/for.html

Profe Mauro Lara. (2020, 14 de diciembre). *4. Tipos de variables - int, long y float* [Video]. YouTube. https://www.youtube.com/watch?v=oc5P019sk1o

Programador Novato. (2022, 12 de agosto). *Ciclo for en Android Studio [31]*. https://www.programadornovato.com/ciclo-for-en-android-studio-31/

Sucesión de Fibonacci. (s. f.). Jardín de los matemáticos, Universidad de Almería. https://www2.ual.es/jardinmatema/sucesion-de-fibonacci/